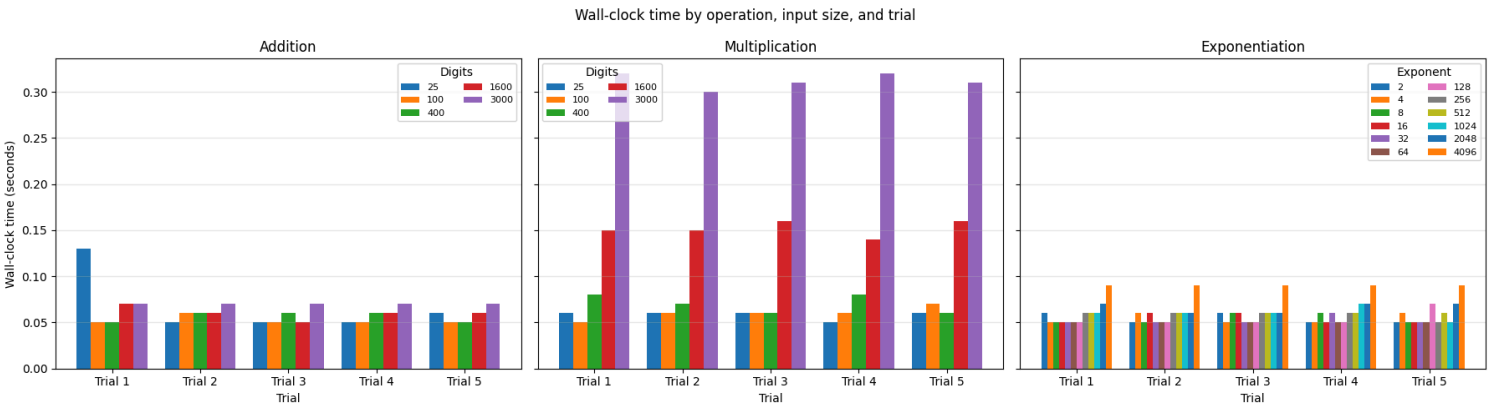
1. Experiment Setup

1. Use `GenOperands` to generate test inputs:
    - addition and multiplication inputs with 25, 100, 400, 1600, and 3000 digits;
    - exponentiation inputs with base 2 and exponents from `e = 2, 4, 8, ..., 4096`.
2. Use `run_measurements.sh` to run every input five times in `--count` mode. For each trial, the script recorded:
    - digit additions
    - digit multiplications
    - node allocations
    - node traversals
    - wall-clock time
3. Full program output and timing logs are saved in `measurement_logs/`. The summarized raw data is saved in `measurements.csv`.
4. The dataset contains 110 measurements: 25 addition, 25 multiplication, and 60 exponentiation trials.

2. Analysis by Topics

2-1. The first run with a small input took much longer, likely due to a cold-start effect.

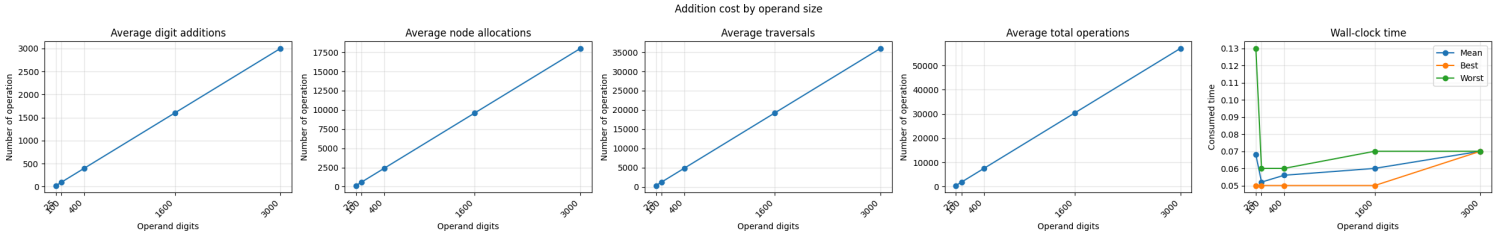
As can be seen from the leftmost figure, the first 25-digit addition trial took longer than the remaining trials. This is most likely a cold-start effect rather than slower arithmetic: each trial starts a new Java process, which may include JVM startup, class loading, operating-system cache warming, and background system variation. The source code was already compiled before measurement, so this is not Java source recompilation. For small inputs, these fixed costs can dominate the actual arithmetic time; therefore, the operation counters provide more reliable evidence of asymptotic cost.



2-1. Time Complexity of Addition

- **Figure:** Every subplot uses the operand digit length on the x-axis. The first four subplots show the average digit additions, node allocations, traversals, and total counted operations across five trials. The final subplot uses running time in seconds and shows the mean, best, and worst trial.
- **Complexity and variable:** Let `n` and `m` be the two operand digit lengths. Addition is `O(max(n, m))`; for this experiment both operands have `N` digits, so the expected cost is `O(N)`.
- **Evidence:** The digit-additions subplot is exactly 25, 100, 400, 1600, and 3000. The node-allocation, traversal, and total-operation subplots increase by approximately the same factor as `N`, supporting linear growth. Wall-clock time is less clear because fixed JVM startup and system overhead dominate small inputs; the first 25-digit trial is a cold-start outlier.

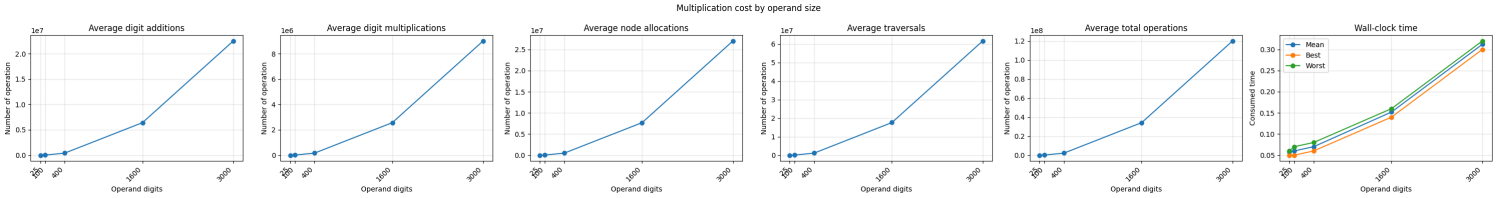
	operand_digits	digit_adds	node_allocs	traversals	wall_seconds
		mean	mean	mean	mean
0	25.0	25.0	155.0	313.0	0.068
1	100.0	100.0	605.0	1213.0	0.052
2	400.0	400.0	2403.0	4809.0	0.056
3	1600.0	1600.0	9605.0	19213.0	0.060
4	3000.0	3000.0	18005.0	36013.0	0.070



2-2. Time Complexity of Multiplication

- **Figure:** Every subplot uses the common operand digit length on the x-axis. The first five subplots show average digit additions, digit multiplications, node allocations, traversals, and total counted operations. The final subplot shows mean, best, and worst running time in seconds.
- **Complexity and variable:** Let `n` and `m` be the digit lengths of the two operands. Grade-school multiplication is `O(nm)`. Because this experiment uses equal-length operands, `n = m = N`, so the expected cost is `O(N^2)`.
- **Evidence:** The digit-multiplications subplot gives 625, 10,000, 160,000, 2,560,000, and 9,000,000, exactly `N^2` at each input size. The remaining counters and total-operations subplot also grow quadratically. The running-time subplot increases most clearly at 1600 and 3000 digits, consistent with the counter evidence.

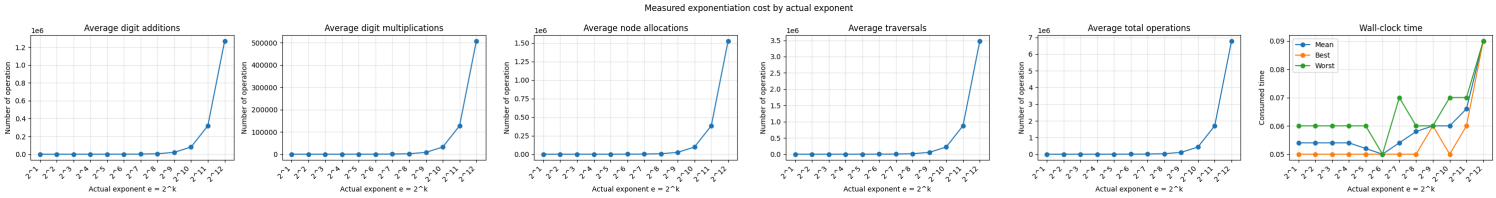
	operand_digits	digit_adds	digit_muls	node_allocs	traversals	wall_seconds
		mean	mean	mean	mean	mean
0	25.0	1538.0	625.0	2013.0	4523.0	0.058
1	100.0	25025.0	10000.0	30669.0	70271.0	0.060
2	400.0	400047.0	160000.0	482592.0	1096545.0	0.070
3	1600.0	6400306.0	2560000.0	7690420.0	17587910.0	0.152
4	3000.0	22497840.0	9000000.0	27017327.0	61589397.0	0.312



2-3. Time Complexity of Exponentiation

- **Figure:**
  - The first figure uses the actual exponent `e` (`2, 4, 8, ..., 4096`) on a log-scale x-axis.
  - The second figure uses `N`, the number of decimal digits in one operand of the final squaring step, `2^(e / 2) * 2^(e / 2)`.
  - In both figures, the first five subplots show mean counter values across five trials. The final subplot shows mean, best, and worst consumed time.
- **Complexity and variable:**
  - Exponentiation by squaring repeatedly halves `e`. Therefore, its control flow has `O(log e)` multiplication levels. The first figure shows the measured cost while `e` doubles at every x-axis step. Although the number of levels grows slowly, the measured counters grow rapidly. Thus, `O(log e)` levels do not mean that the digit-level cost is `O(log e)`.
  - To explain this growth, the second figure rewrites the same measurements using `N`, the digit length of the operands in the final and largest multiplication. The multiplication experiment showed that grade-school multiplication costs `O(N^2)`. The final multiplication is the largest and dominates the smaller earlier multiplications, so exponentiation has measured digit-level cost `O(N^2)`.
  - The base is fixed at 2, so `N` grows proportionally to the numerical exponent `e`. Therefore, `O(N^2)` is equivalently `O(e^2)` in this experiment.

	exponent	digit_adds	digit_muls	node_allocs	traversals	wall_seconds
		mean	mean	mean	mean	mean
0	2.0	2.0	1.0	11.0	24.0	0.054
1	4.0	5.0	2.0	18.0	39.0	0.054
2	8.0	14.0	6.0	31.0	71.0	0.054
3	16.0	37.0	15.0	64.0	150.0	0.054
4	32.0	102.0	40.0	151.0	357.0	0.052
5	64.0	355.0	140.0	471.0	1115.0	0.050
6	128.0	1353.0	540.0	1692.0	3933.0	0.054
7	256.0	5164.0	2061.0	6317.0	14581.0	0.058
8	512.0	20340.0	8145.0	24599.0	56319.0	0.060
9	1024.0	80345.0	32170.0	96761.0	220625.0	0.060
10	2048.0	319003.0	127651.0	383476.0	876705.0	0.066
11	4096.0	1270768.0	508340.0	1526383.0	3483230.0	0.090



	exponent	final_factor_digits
9	1024.0	155

